arXiv:2306.00110v1 [cs.SD] 31 May 2023

MuseCoco: Generating Symbolic Music from Text

Peiling Lu[‡*], **Xin Xu**[♭*], **Chenfei Kang**[♯*], **Botao Yu**[♮*], **Chengyi Xing**[×*], **Xu Tan**[‡†], **Jiang Bian**[‡]
[‡]Microsoft Research Asia
[♭]Zhejiang University, [♯]Shanghai Jiao Tong University
[♮]Nanjing University, [×]Dalian University of Technology
{peil, xuta, jiabia}@microsoft.com
xxucs@zju.edu.cn, chenfeikang314@gmail.com
btyu@foxmail.com, xcyhbp@mail.dlut.edu.cn
https://github.com/microsoft/muzic

Abstract

Generating music from text descriptions is a user-friendly mode since the text is a relatively easy interface for user engagement. While some approaches utilize texts to control music audio generation, editing musical elements in generated audio is challenging for users. In contrast, symbolic music offers ease of editing, making it more accessible for users to manipulate specific musical elements. In this paper, we propose MuseCoco, which generates symbolic music from text descriptions with musical attributes as the bridge to break down the task into text-to-attribute understanding and attribute-to-music generation stages. MuseCoCo stands for Music **Co**mposition **Co**pilot that empowers musicians to generate music directly from given text descriptions, offering a significant improvement in efficiency compared to creating music entirely from scratch. The system has two main advantages: Firstly, it is data efficient. In the attribute-to-music generation stage, the attributes can be directly extracted from music sequences, making the model training self-supervised. In the text-to-attribute understanding stage, the text is synthesized and refined by ChatGPT based on the defined attribute templates. Secondly, the system can achieve precise control with specific attributes in text descriptions and offers multiple control options through attribute-conditioned or text-conditioned approaches. MuseCoco outperforms baseline systems in terms of musicality, controllability, and overall score by at least 1.27, 1.08, and 1.32 respectively. Besides, there is a notable enhancement of about 20% in objective control accuracy. In addition, we have developed a robust large-scale model with 1.2 billion parameters, showcasing exceptional controllability and musicality. Music samples generated by MuseCoco are available via this link [1], and the code is available at this link [2].

1 Introduction

Text-to-music generation is an important task in automatic music generation because it allows users to generate music more easily and intuitively, using natural language as an interface. This makes it a user-friendly mode of music generation, particularly for those who do not have a background in music theory or composition. Previous work [1–4] generating musical audio from texts faces the

[*]These authors contributed equally to this work.
[†]Correspondence: Xu Tan, xuta@microsoft.com
[1]https://ai-muzic.github.io/musecoco/
[2]https://github.com/microsoft/muzic/musecoco/

Preprint. Under review.

following challenges: 1) Limited adaptability: musical audio generated from texts is less adaptable than symbolic music. Once the audio is produced, it may be difficult to make significant changes to the music without starting the generation process over. 2) Lack of control: generating musical audio from text descriptions lacks control over specific aspects of the music such as tempo, meter, and rhythm since the generation process is not explicitly controlled. However, they can be easily handled by generating symbolic music from given texts. Symbolic music refers to a type of music notation that uses symbols and music language to represent specific music ideas, thus, it is more adaptable. Besides, specific musical attributes can be extracted from symbolic data, which can enable more precise control.

There are some works [5–7] attempt to generate symbolic music from text descriptions, but they also face issues with unnatural text descriptions and poor performance. Limitations exist in certain approaches [5] where the model is restricted to specific textual inputs, hindering its ability to generalize text representations to a more user-friendly format. Besides, some work [5] can only control music generation based on limited music aspects, which limits the model's ability to capture the full range of musical creativity and does not satisfy the requirements of all users. While certain approaches [6, 7] allow for natural language inputs, their control accuracy and musical quality is limited due to the requirement for large amounts of paired text-music data. Mubert[3] can produce editable MIDI compositions, but it merely retrieves and combines pre-existing music pieces rather than generating novel ideas, limiting its responsiveness to input prompts.

We propose MuseCoco, a system for generating symbolic music from text descriptions by leveraging musical attributes. MuseCoCo, stands for Music **Co**mposition **Co**pilot, is a powerful tool that empowers musicians to generate music directly from provided text descriptions. By harnessing the capabilities of MuseCoCo, musicians experience a substantial increase in efficiency, eliminating the need to create music entirely from scratch. Our approach breaks down the task into two stages: text-to-attribute understanding and attribute-to-music generation stage. Musical attributes can be easily extracted from music sequences or obtained from existing attribute-labeled datasets, allowing the model in the attribute-to-music generation stage to be trained in a self-supervised manner. Musical attributes can also be inferred from natural languages in the text-to-attribute understanding stage. To synthesize paired text-to-attribute data, templates are created for each attribute, and a subset of these templates is combined and further refined into a coherent paragraph using ChatGPT's language generation capabilities.

With self-supervised training in attribute-to-music generation and supervised learning via the help of synthesized paired data in text-to-attribute understanding, a large amount of symbolic music data can be leveraged without the need of providing textual prompts manually. This can help improve model performance by increasing data amount and model size simultaneously. Besides, by leveraging various music attributes, explicit control can be achieved in generating music across multiple aspects, enabling fine-grained manipulation and customization of the generated musical output. Moreover, due to the two-stage design, MuseCoco can support multiple ways of controlling. For instance, musicians with a strong knowledge of music can directly input attribute values into the second stage to generate compositions, while users without a musical background can rely on the first-stage model to convert their intuitive textual descriptions into professional attributes. Thus, MuseCoco allows for a more inclusive and adaptable user experience than those systems that directly generate music from text descriptions.

The main contributions of this work are as follows:

- We introduce MuseCoco, a system that seamlessly transforms textual input into musically coherent symbolic compositions. This innovative approach empowers musicians and general users from diverse backgrounds and skill levels to create music more efficiently and with better control.
- With this two-stage framework, a large amount of symbolic data can be used without the need for labeled text descriptions. It offers users two engagement options: directly specifying attribute values or leveraging text descriptions to control the music generation process.
- Subjective evaluation results have demonstrated that MuseCoco outperforms baseline systems in terms of musicality, controllability, and overall score, achieving a minimum improvement of 1.27, 1.08, and 1.32, respectively. Additionally, there is a significant boost of about 20% in objective control accuracy, further affirming the system's enhanced performance.

[3] `https://mubert.com/`

- We also extend our model to a large scale with 1.2 billion parameters, which exhibits notable controllability and musicality, further enhancing its performance.

2 Related Work

2.1 Text-to-Music Generation

Text-to-music generation based on deep learning has been an active research area. Though paired text-music data is scarce, there have been many works for generating audio music from input prompts. A database containing music created by musicians and sound designers with three tags (genres, moods and activities) is constructed in MubertAI[4]. It assigns the closest tags for the input prompt and generates combination of sounds from the database based on these tags. Riffusion[5] leverages stable diffusion [8] to obtain images of music spectrograms paired with input text and produce audio from them. Meanwhile, both Moûsai [4] and ERNIE-Music [3] apply diffusion models with self-collected text-audio datasets to achieve text-audio music generation. Recently, Huang et al. [9] create a large audio-text dataset to train a joint embedding model linking music audio and natural language music descriptions, MuLan, which helps address the absence of text-music paired data. For example, in Noise2Music [2], MuLan assigns captions to unlabeled audio clips to create music-text pairs. By directly using input text representation from MuLan during inference, MusicLM [1] can be trained to output audio music from Mulan audio representation on existing audio data to complete the text-to-music generation task without paired text-audio data. However, the challenge with musical audio generation from texts lies in its limited editability, which poses limitations in the composition process. By contrast, symbolic music, notated as a sequence of musical symbols, can be simply interpreted and manipulated by humans and machines.

Only a few works focus on generating symbolic music from text descriptions. BUTTER [5], a music-sentence representation learning framework, proposes music representation disentanglement and cross-modal alignment to firstly generate music presented by ABC notations[6] from text including four musical keywords (the key, meter, style, and others). Limited to the folk song datasets and a few musical factors, it cannot generate symbolic music in many varieties. Instructed with musical input prompts, the large language model GPT-4 [7] can also generate ABC notation music, however, without any nontrivial form of harmony [10]. Wu and Sun [6] also explores the text-to-music capability of pre-trained language models. Though fine-tuned with more than 200k text and ABC notation music pairs, the model cannot align the musical attribute values in loose text with the generated music well enough and can only generate music with solo tracks.

Previous work directly generates music from text descriptions, which lacks explicit control over the generation process. In this paper, we propose MuseCoco, which can generate symbolic music from text descriptions with high control accuracy and good musicality. The generated music, in its symbolic format, offers easy editability and can be explicitly controlled through attribute values derived from text descriptions.

2.2 Controllable Music Generation

Controllable music generation refers to the ability to exert control over specific aspects or characteristics of the generated music. By applying conditional generative models (such as conditional VAE[11–13], GAN[14, 15] or diffusion models[2, 3]), previous work leverage different conditions to generate music: Some leverage descriptions like emotion [16–18], style [19–21] or structure[22, 23] as conditions to control music generation. The other applies music descriptions such as instrumentation[24, 25], chord[11, 26], note density [12, 13], etc. to generate music.

However, general users usually have multiple requirements for generated music, and previous work with limited control over specific music aspects may result in limited expressiveness and diminished adaptability to different musical contexts. Besides, previous work can only control music generation with specified music attributes, which restricts the model ability to convey complex and nuanced musical ideas or concepts that can be effectively communicated through textual descriptions. Text-

[4] https://github.com/MubertAI/Mubert-Text-to-Music

[5] https://www.riffusion.com/about

[6] https://abcnotation.com/

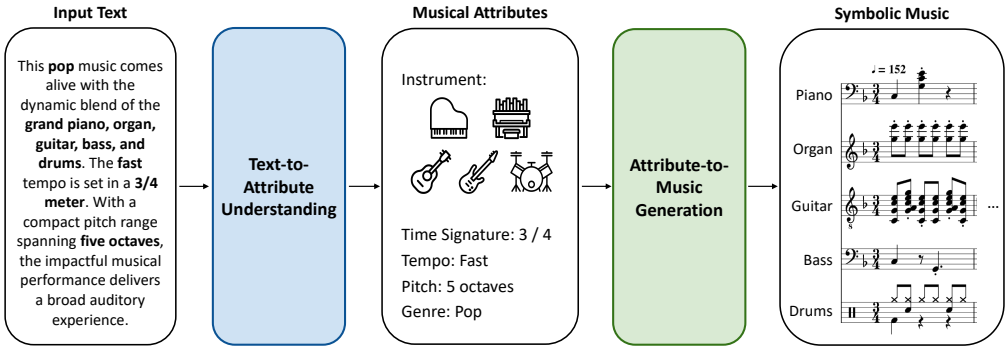


Figure 1: The two-stage framework of MuseCoco. Text-to-attribute understanding extracts diverse musical attributes, based on which symbolic music is generated through the attribute-to-music generation stage.

based input is easily accessible and familiar to users, making it a practical and widely adopted choice for guiding generative tasks, such as text-to-image[8, 27–29] and text-to-music generation[2, 1, 6, 10]. Previous work has struggled to generate symbolic music directly from textual descriptions provided by users. MuseCoco maps text input to music attributes and then utilizes music attributes to control music generation, which can effectively solve the above problems.

3 MuseCoco

3.1 Overview

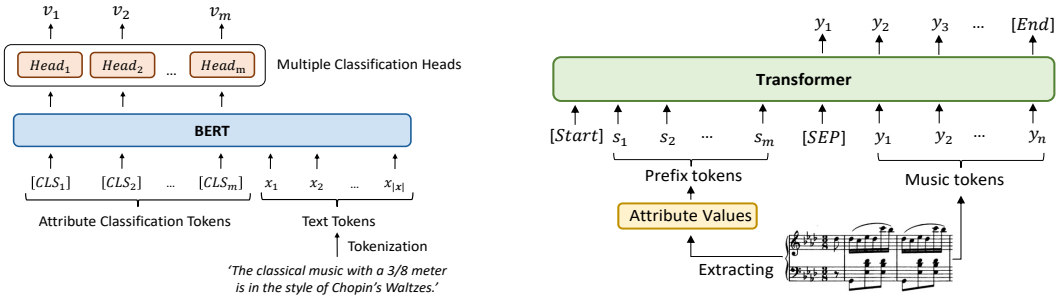
To achieve text-to-music controllable generation, MuseCoco incorporates natural language and symbolic music into a two-stage framework that separates text-to-attribute understanding and attribute-to-music generation, which are trained independently. The pipeline of MuseCoco is shown in Figure 1. In this section, we will elaborate on their technical design and model architectures respectively.

3.2 Attribute-to-Music Generation

Most musical attributes can be easily obtained by extracting from music sequences (Section 4.1), so the music generation model in the attribute-to-music stage can be trained in a self-supervised way. This method can leverage large amounts of unlabeled data, making it a highly data-efficient approach. Musical attributes (Table 1) can be classified into objective attributes like the tempo and meter and subjective attributes like the emotion and genre [30]. Objective attributes refer to quantifiable and measurable characteristics of musical elements, so they can be extracted from music sequences with pre-defined rules (please refer to Section 4.1 for more details). Subjective attributes refer to the qualities or characteristics of music that are based on personal interpretation, perception, or emotional response, which can be obtained from existing attribute-labeled datasets. After obtaining attributes from music sequences, we append those attribute tokens as prefix tokens into music sequences to provide explicit control over the music, which makes it easier to interpret and understand how the attributes are influencing the music. Different attribute tokens can be combined or sequenced to achieve complex musical expressions and transformations.

Specifically, given dataset $\{\mathcal{V}, \mathcal{Y}\}$, where $\mathcal{Y}$ is the symbolic music dataset and $\mathcal{V}$ is the set of attribute values, MuseCoco transforms attribute values into prefix tokens to control music generation, as shown in Figure 2(b). Using prefix tokens to guide the generation process is an effective method for directing the output towards a particular direction [31–34]. We pre-define m musical attributes and their values as shown in Appendix A. For each music sequence $\boldsymbol{y} = [y_1, y_2, ..., y_n] \in \mathcal{Y}$, and its attribute values $\boldsymbol{v} = \{v_1, v_2, ..., v_m\} \in \mathcal{V}$, we consider the following distribution:

$$p(\boldsymbol{y}|\boldsymbol{v}) = \prod_{i=1}^{n} p(y_i|y_{<i}, v_1, v_2, ..., v_m). \tag{1}$$



(a) Text-to-attribute understanding

(b) Attribute-to-music generation

Figure 2: During training, each stage is independently trained. During inference, the text-to-attribute understanding stage firstly extracts music attribute values, based on which the attribute-to-music generation stage secondly generates symbolic music.

We encode each attribute value v_j into a prefix token s_j. Position embeddings for such prefix tokens $\{s_j\}_{j=1}^{m}$ are not used. Since it is not usual for users to provide all attribute values in real-world scenarios, we introduce a special token, represented by s_j^{NA}, to exclude this attribute (i.e. v_j) that is not specified in inputs from influencing the music generation process. Then the input sequence is encoded into:

$$s_1, s_2, s_3, ..., s_m, \texttt{[SEP]}, y_1, y_2, ..., y_n. \tag{2}$$

During training, some attribute tokens (e.g., s_2, s_3) are randomly replaced with special tokens (i.e., s_j^{NA}) to enable adaptation to various attribute combinations. During inference, attribute values are provided directly from inputs. Any attributes that are absent in the inputs are represented by special tokens, which are combined with the other prefix tokens to effectively control the music generation process as required.

3.3 Text-to-Attribute Understanding

Control information within input text comprises different musical attributes. Therefore, the text-to-attribute task is required to extract musical attribute values from plain text. These attribute values will be used in the attribute-to-music generation stage to generate desired music. As shown in Figure 2(a), this stage can be denoted as $\{\mathcal{X}, \mathcal{V}\}$, where $\mathcal{X}$ is the input text set, $\mathcal{V}$ is the value set of m pre-defined musical attributes. In the dataset, each instance $\boldsymbol{x} \in \mathcal{X}$ is paired with a combination of m attribute values $\boldsymbol{v} = \{v_i\}_{i=1}^{m}$, $\boldsymbol{v} \in \mathcal{V}$. Given a pre-trained language model $\mathcal{M}$, $\text{BERT}_{\text{large}}$ [35], $\boldsymbol{x}$ is converted by the tokenizer of $\mathcal{M}$ into corresponding tokens $\{x_1, x_2, \ldots, x_{|\boldsymbol{x}|}\}$. To adapt to multiple attribute classification, we prepend m attribute classification tokens $\texttt{[CLS}_i\texttt{]}_{i=1}^{m}$ to input text tokens as m attribute classification heads and the encoded $\texttt{[CLS}_i\texttt{]}$ head is used to compute the probability distribution over the i class set with a softmax classifier. Position embeddings of $\texttt{[CLS}_i\texttt{]}_{i=1}^{m}$ are not used to be consistent with the pre-training stage. Specifically, the input x will be encoded to hidden vectors:

$$\mathbf{h}_{\texttt{[CLS}_1\texttt{]}}, \mathbf{h}_{\texttt{[CLS}_2\texttt{]}}, \ldots, \mathbf{h}_{\texttt{[CLS}_m\texttt{]}}, \mathbf{h}_{x_1}, \mathbf{h}_{x_2}, \ldots, \mathbf{h}_{x_{|\boldsymbol{x}|}}$$

Table 1: Musical attribute descriptions.

Type	Attribute	Description
Objective	Instrument	played instruments in the music clip
	Pitch	the number of octaves covering all pitches in one music clip
	Rhythm Danceability	whether the piece sounds danceable
	Rhythm Intensity	the intensity of the rhythm
	Bar	the total number of bars in one music clip
	Time Signature	the time signature of the music clip
	Key	the tonality of the music clip
	Tempo	the tempo of the music clip
	Time	the approximate time duration of the music clip
Subjective	Artist	the artist (style) of the music clip
	Genre	the genre of the music clip
	Emotion	the emotion of the music clip

Table 2: An example of synthesizing a text-attribute pair. We randomly select a template from the available templates for each attribute. Here shows two of each. Then the templates are refined by ChatGPT and then filled in values.

Attribute	Value	Template
Key	Major	This music is composed in the [KEY] key. This music's use of [KEY] key creates a distinct atmosphere.
Bar	13 ~ 16	The song is composed of approximately [NUM_BARS] bars. The song comprises [NUM_BARS] bars.
Emotion	Happiness	The music is imbued with [EMOTION]. The music has a [EMOTION] feeling.
Time Signature	4 / 4	The [TIME_SIGNATURE] time signature is used in the music. The music is in [TIME_SIGNATURE].
Refine via ChatGPT and fill in placeholders with values: {Key, Bar, Emotion, Time Signature}		The music is imbued with happiness, and the major key in this music provides a powerful and memorable sound. The song progresses through 13 ~ 16 bars, with 4/4 as the meter of the music.

The probability distribution of the value of i-th attribute is $p_i(v_i|\boldsymbol{x}) = \texttt{Softmax}(\mathbf{W_i}\mathbf{h}_{\texttt{[CLS}_i\texttt{]}} + \mathbf{b_i})$, where $\mathbf{W_i}$ and $\mathbf{b_i}$ are learnable parameters. The cross-entropy loss of i-th attribute is

$$\mathcal{L}_i = -\frac{1}{|\mathcal{X}|}\sum_{\boldsymbol{x}\in\mathcal{X}} \log p_i(v_i|\boldsymbol{x})$$

During training, all learnable parameters are fine-tuned by minimizing the sum of attribute cross-entropy losses $\mathcal{L} = \sum_{i=1}^{m} \mathcal{L}_i$ on $\{\mathcal{X}, \mathcal{V}\}$.

3.4 Data Construction

By using musical attributes to break down the text-to-music generation task into the attribute-to-music generation stage and the text-to-attribute understanding stage, we can leverage large amounts of symbolic music data without text descriptions. In the attribute-to-music generation stage, attributes can be extracted from music sequences with rules or obtained from attribute-labeled datasets (detailed in Section 4.1). The system only requires paired data in the text-to-attribute stage, we synthesize these text-attribute pairs in the following steps:

1. **Write templates for each attribute:** As shown in Table 2, we write several templates as a set for each attribute, where its values are represented with a placeholder. By utilizing this placeholder, we can accommodate diverse combinations of attribute values without requiring exact values.
2. **Create attribute combinations and concatenate their templates as paired texts:** The generation process is usually controlled by multiple attributes together. Hence, constructing various different combinations of attribute values and paired text is necessary. Because of the long-tail distribution per attribute value in real-world data, to enrich the diversity of paired text-attribute training data and avoid the long-tailed issue, we stochastically create $\boldsymbol{v}$ per instance based on pre-defined musical attributes and their values on our own to ensure the number of instances including v_i is balanced, i.e., each value of each attribute is sampled equally. And then, the paired texts of these created combinations are synthesized by simply concatenating their corresponding templates, randomly chosen from template sets of each attribute.
3. **Refine concatenated templates via ChatGPT:** Since simply concatenated templates are less fluent than real users' input, ChatGPT[7] is utilized to refine them as shown in Table 2.
4. **Fill in placeholders:** Finally, attribute values or their synonyms fill in placeholders, ensuring that the resulting text effectively conveys the intended meaning and maintains a consistent narrative structure.

Through these steps, we can independently construct the datasets for either of the two stages without the need for paired text-music data.

[7] `https://chat.openai.com/`

4 Experiments

4.1 Experiment Setup

Datasets To train the attribute-to-music generation stage and evaluate our proposed method, we collect an assortment of MIDI datasets from online sources. Table 3 lists all of the used datasets along with their respective counts of valid MIDI files. Specifically, the MMD dataset [36] consists of many datasets collected from the internet[8]. The Emotion-gen dataset is generated by our internal emotion-controllable music generation system, and the others are all publicly released datasets. We did the necessary data filtering to remove duplicated and poor-quality samples, and there are 947,659 MIDI samples remaining. From each MIDI file, we randomly extracted 3 clips within 16 bars. The attributes described in Table 1 were then extracted from each clip. The objective attribute values used in the training are directly extracted from MIDI files and the subjective attribute values are obtained from some of the datasets (details in Appendix A).

Table 3: Statistics of the used datasets.

Dataset	#MIDI
MMD [36]	1,524,557
EMOPIA [16]	1,078
MetaMidi[37]	612,088
POP909 [38]	909
Symphony [39]	46,360
Emotion-gen	25,730
Total (after filtering)	947,659

System Configuration For the attribute-to-music generation stage, we use a REMI-like [40] representation method to convert MIDI into token sequences. We apply Linear Transformer[41] as the backbone model, which consists of 16 layers with causal attention and 12 attention heads. The hidden size is 1024 and FFN hidden size is 4096, yielding an approximate parameter count of 203 million. The max length of each sample is 5120, covering at most 16-bar music segments. During training, the batch size is 64. The dropout rate is set to 0.1. We use Adam optimizer [42] with $\beta_1 = 0.9, \beta_2 = 0.98$ and $\epsilon = 10^{-9}$. The learning rate is 2×10^{-4} with warm-up step 16000 and an invert-square-root decay. In text-to-attribute understanding, we leverage $\text{BERT}_{\text{large}}$[9] as the backbone model and the max sequence length of it is set to 256, which covers common user input. We use 1,125 thousand samples for fine-tuning with a train/valid portion of 8:1. During training, the batch size is 64 and the learning rate is 1×10^{-5}.

Evaluation Dataset and Metrics To evaluate MuseCoco, we construct a **standard test set** including 5,000 text-attribute pairs in the same way in Section 3.4. Musical attributes of each test sample originated from real music in the test set of the attribute-to-music stage, instead of creating them on our own to make sure musical rationality and all values of the attributes are covered in the test set for thorough testing. Meanwhile, in order to accord with usual user inputs, we randomly assign the *NA* value (meaning the attribute is not mentioned in the text) to some attributes per sample to synthesize text prompts with different lengths.

To evaluate the models objectively, we propose the metric called Average Sample-wise Accuracy (ASA), which is calculated by determining the proportion of correctly predicted attributes in each sample, followed by the calculation of the average prediction accuracy across the whole test set. To conduct a subjective evaluation of MuseCoco's performance, we employ a user study. We invite individuals with musical backgrounds to fill out the questionnaires (details in Appendix B.1) Participants are asked to rate the following metrics on a scale of 1 (lowest) to 5 (highest):

- **Musicality:** This metric assesses the degree to which the generated music exhibits qualities akin to the artistry of a human composer.
- **Controllability:** This metric measures how well the samples adhere to the musical attribute values specified in the text descriptions.
- **Overall:** This metric quantifies the overall quality of this generated music considering both its musicality and controllability.

[8] We obtained the dataset for this work with the help of the authors, as it was not publicly available.

[9] `https://huggingface.co/bert-large-uncased`

4.2 Comparison with Baselines

Baselines In this study, we compare our method to two existing works for generating symbolic music from 21 text descriptions randomly selected from the standard test set:

- **GPT-4**: GPT-4 [7] is a large-scale language model that demonstrated its capabilities in various domains, including music. Following Bubeck et al. [10], we instruct GPT-4 to generate ABC notation music with the task-specific prompts (in Appendix B.5) using the official web page[10] manually.
- **BART-base**: Wu and Sun [6] release a language-music BART-base[11], which shows a solid performance. Text descriptions are fed into this model and guide it to generate ABC notation music for comparison.

For a fair comparison, we convert ABC notation music generated by baselines into MIDI music using music21[12]. As for the subjective evaluation, well-designed questionnaires including generated music from baselines and our method are distributed to individuals, who are all in music backgrounds and required to score the subjective metrics described in Section 4.1 (details in Appendix B.3). Meanwhile, to objectively compare the model ability, we calculate the average sample-wise accuracy of generated music for both baselines and our method (details in Appendix B.2).

Main Results Table 4 reports the main results of the comparison. In terms of musicality, MuseCoco achieves the highest score (mean 4.06 out of 5), indicating its ability to generate music closely resembling compositions by humans and approaching the quality of real-world music. As for the conditional generation ability, MuseCoco outperforms all baselines in terms of controllability and the average sample-wise accuracy at 1.08 and 19.95% respectively, which illustrates the effectiveness of controlling musical attributes with the two-stage framework. GPT-4 also can generate more coherent music with input prompts than BART-base due to its powerful ability of language understanding. Meanwhile, the best overall score (mean 4.13 out of 5) of MuseCoco shows our method is capable of generating the most favorite and fair-sounding music through the auditory test. Music samples generated by MuseCoco are available via this link[13].

Table 4: Comparison between MuseCoco, GPT-4, and BART-base. ASA stands for the average sample-wise accuracy.

	Musicality	Controllability	Overall	ASA (%)
MuseCoco	**4.06** $\pm$ 0.82	**4.15** $\pm$ 0.78	**4.13** $\pm$ 0.75	**77.59**
GPT-4 [7]	2.79 $\pm$ 0.97	3.07 $\pm$ 1.05	2.81 $\pm$ 0.97	57.64
BART-base [6]	2.19 $\pm$ 1.14	2.02 $\pm$ 1.09	2.17 $\pm$ 1.03	31.98

4.3 Method Analysis

In this section, we conduct analysis experiments on the two stages respectively.

4.3.1 Analysis on Text-to-Attribute Understanding

Attribute Comprehension To evaluate the ability to extract each attribute from text, we test the text-to-attribute model on the standard test set and show the classification accuracy of each attribute in Appendix A. Each accuracy consistently surpasses 99%, which proves that the model exhibits exceptional performance on all attributes and showcases the high accuracy and reliability of the text understanding ability.

Different classification heads We explore the effectiveness of using multiple classification heads and report the ASA shown in Table 5. The model with multiple classification heads outperforms the

[10] `https://chat.openai.com/`
[11] `https://huggingface.co/sander-wood/text-to-music`
[12] `http://web.mit.edu/music21/`
[13] `https://ai-muzic.github.io/musecoco/`

one-head BERT by 39.87%, which illustrates that each head can learn their corresponding attribute knowledge, and using multiple heads can improve the overall performance.

Different text synthetic strategies In order to showcase the effectiveness of the refinement strategy outlined in Section 3.4, we evaluate its impact on enhancing the fluency and diversity of the text within the training set during the text-to-attribute stage. We engage musicians to write 17 text descriptions manually to help evaluate and contrast the effectiveness of synthesis strategies w/ and w/o the refinement step. As shown in Table 6, we observe that fine-tuning the model with 25% partially refined text achieves better performance than with the simply concatenated templates.

Table 5: Analysis on multiple classification heads.

	ASA (%)
One Head	60.09
Multiple Classification Heads	99.96

Table 6: Analysis on text refinement.

	ASA (%)
Concatenated	75.88
Concatenated + Refined	78.11

4.3.2 Analysis on Attribute-to-Music Generation

Attribute Control Performance To evaluate the controllability of the attribute-to-music generation model, we report the control accuracy results for each attribute in Appendix B.4. The average attribute control accuracy is 80.42%, demonstrating a strong capability of the model to effectively respond to the specified attribute values during the music generation process.

Study on Control Methods We compare *Prefix Control*, which is the default method of our model that uses prefix tokens to control music generation, with two other methods: 1) *Embedding*: Add attribute input as embedding to token embedding; 2) *Conditional LayerNorm*: Add attribute input as a condition to the layer norm layer [43, 44]. We utilize Musicality and average attribute control accuracy as evaluation metrics. For more details on this experiment, please refer to the description in Appendix B.4. We report evaluation results in Table 7. It is shown that *Prefix Control* outperforms other methods in terms of musicality and average attribute control accuracy, with a minimum improvement of 0.04 and 19.94% respectively, highlighting its superior capability to capture the relationship between attributes and music.

Table 7: Comparison of different control methods. Musicality reflects the quality of the generated music. Average attribute control accuracy represents the control accuracy over all attributes, which can reflect controllability.

Method	Musicality ↑	Average attribute control accuracy (%) ↑
Embedding	2.97 ± 0.91	36.94
Conditional LayerNorm	3.11 ± 1.02	47.46
Prefix Control	$\mathbf{3.15} \pm 1.02$	**67.40**

Study on Model Size We conduct a comparative analysis between two different model sizes to determine whether increasing the model size would result in improved generated results. The parameter configurations for these model sizes are presented in Table 8. The default model, referred to as *large*, is the default model for the attribute-to-music generation stage. Additionally, we utilize *xlarge* model for comparison, which consists of approximately 1.2 billion parameters. The training of *xlarge* model follows the same settings outlined in Section 4.1. The objective evaluation results are displayed in Table 8, which indicates that increasing the model size enhances controllability. To further evaluate the performance, we will conduct subjective listening tests to compare the subjective evaluation results for these two model sizes.

4.3.3 Comments from Musicians

We invite professional musicians to give their comments on generated music samples from given texts based on our system. Here are some feedbacks:

The generated music closely resembles human compositions, displaying a high level of accuracy and creativity. It provides inspiration for creative compositions, showcasing interesting motifs and

Table 8: Comparison of different model sizes in the attribute-to-music generation stage. Average objective attribute control accuracy represents the control accuracy over objective attributes, which can reflect controllability.

Model Size	Layers	d_{model}	Parameters	Average objective attribute control accuracy (%) ↑
large	16	1024	203M	83.63
xlarge	24	2048	1.2B	**87.15**

demonstrating skillful organization of musical elements. This work greatly improves composition efficiency, saving approximately one day of time. —- from Musician A

The generated music offers arrangement inspiration, exemplified by the idea of combining right-hand arpeggios and left-hand bass melody to facilitate creative expansion in composition. It sparks the concept of blending classical and popular music genres described in texts. —- from Musician B

The generated music significantly reduces my composition time, saving anywhere from 2 days to 2 weeks, particularly when it comes to instrumental arrangements that are outside my familiarity. The composition incorporates inspirational elements in some sections of the piece. For example, I very much enjoyed the journey through the conflicts and resolutions. In some areas towards the end it felt like I was embarking on an adventure up a mountain and through grassy fields, very interesting. —- from Musician C

The generated music presents the various elements of the text well, which provides great convenience for musicians to edit the music. For example, music in the style of Bach is well-created, and multiple instruments in the generated music are arranged harmoniously. Meanwhile, as for music teachers, MuseCoco is very helpful in generating desired musical examples for education. —- from Musician and Music Teacher D

The feedback from musicians has demonstrated our ability to enhance their workflow efficiency by reducing redundant tasks and providing creative inspiration.

5 Conclusion

In conclusion, this paper makes several significant contributions to the field of music generation and the application of AI in creative tasks. We introduce MuseCoco, a system that seamlessly transforms text descriptions into musically coherent symbolic compositions. This innovative approach empowers musicians and general users from diverse backgrounds and skill levels to create music more efficiently and with greater control.

Second, we present a two-stage design that simplifies the learning process and enhances controllability. This design reduces the reliance on large amounts of paired text-music data and improves more explicit control through various aspects of attributes. By leveraging a large amount of symbolic music data, we achieve impressive musicality and establish a coherent connection between text descriptions and the generated music.

Our research demonstrates the potential of AI technologies in facilitating idea generation and streamlining the composition process for music creation tasks. By offering a powerful and adaptable tool like MuseCoco, we aim to inspire and empower artists to overcome the challenges they face in their creative pursuits and unlock new possibilities in music composition. The utilization of generative AI for creative purposes often raises concerns regarding copyright and ownership, which necessitates careful consideration moving forward. Limitations of our work can be seen in Appendix C.

6 Acknowledgment

We would like to express our sincere gratitude to Amy Sorokas for her invaluable help in connecting with musicians. We would also like to thank musician, Grentperez, and musicians from Central Conservatory of Music in China for the collaboration and constructive feedback, which contributed to the success of this work. Meanwhile, we sincerely appreciate all members of Wenqin Piano Society from Zhejiang University for completing the most questionnaires.

References

[1] A. Agostinelli, T. I. Denk, Z. Borsos, J. Engel, M. Verzetti, A. Caillon, Q. Huang, A. Jansen, A. Roberts, M. Tagliasacchi *et al.*, "Musiclm: Generating music from text," *arXiv preprint arXiv:2301.11325*, 2023.

[2] Q. Huang, D. S. Park, T. Wang, T. I. Denk, A. Ly, N. Chen, Z. Zhang, Z. Zhang, J. Yu, C. Frank *et al.*, "Noise2music: Text-conditioned music generation with diffusion models," *arXiv preprint arXiv:2302.03917*, 2023.

[3] P. Zhu, C. Pang, S. Wang, Y. Chai, Y. Sun, H. Tian, and H. Wu, "Ernie-music: Text-to-waveform music generation with diffusion models," *arXiv preprint arXiv:2302.04456*, 2023.

[4] F. Schneider, Z. Jin, and B. Schölkopf, "Moûsai: Text-to-music generation with long-context latent diffusion," *arXiv preprint arXiv:2301.11757*, 2023.

[5] Y. Zhang, Z. Wang, D. Wang, and G. Xia, "BUTTER: A representation learning framework for bi-directional music-sentence retrieval and generation," in *Proceedings of the 1st Workshop on NLP for Music and Audio (NLP4MusA)*. Association for Computational Linguistics, 16 Oct. 2020, pp. 54–58.

[6] S. Wu and M. Sun, "Exploring the efficacy of pre-trained checkpoints in text-to-music generation task," *arXiv preprint arXiv:2211.11216*, 2022.

[7] OpenAI, "Gpt-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.

[8] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-resolution image synthesis with latent diffusion models," in *CVPR*. IEEE, 2022, pp. 10 674–10 685.

[9] Q. Huang, A. Jansen, J. Lee, R. Ganti, J. Y. Li, and D. P. W. Ellis, "Mulan: A joint embedding of music audio and natural language," *arXiv preprint arXiv:2208.12415*, 2022.

[10] S. Bubeck, V. Chandrasekaran, R. Eldan, J. Gehrke, E. Horvitz, E. Kamar, P. Lee, Y. T. Lee, Y. Li, S. Lundberg *et al.*, "Sparks of artificial general intelligence: Early experiments with gpt-4," *arXiv preprint arXiv:2303.12712*, 2023.

[11] Z. Wang, D. Wang, Y. Zhang, and G. Xia, "Learning interpretable representation for controllable polyphonic music generation," *arXiv preprint arXiv:2008.07122*, 2020.

[12] H. H. Tan and D. Herremans, "Music fadernets: Controllable music generation based on high-level features via low-level feature modelling," *arXiv preprint arXiv:2007.15474*, 2020.

[13] D. von Rütte, L. Biggio, Y. Kilcher, and T. Hofmann, "Figaro: Controllable music generation using learned and expert features," in *The Eleventh International Conference on Learning Representations*, 2023.

[14] P. Neves, J. Fornari, and J. Florindo, "Generating music with sentiment using transformer-gans," *arXiv preprint arXiv:2212.11134*, 2022.

[15] Y. Zhu, K. Olszewski, Y. Wu, P. Achlioptas, M. Chai, Y. Yan, and S. Tulyakov, "Quantized gan for complex music generation from dance videos," in *Computer Vision–ECCV 2022: 17th European Conference, Tel Aviv, Israel, October 23–27, 2022, Proceedings, Part XXXVII*. Springer, 2022, pp. 182–199.

[16] H.-T. Hung, J. Ching, S. Doh, N. Kim, J. Nam, and Y.-H. Yang, "Emopia: A multi-modal pop piano dataset for emotion recognition and emotion-based music generation," *arXiv preprint arXiv:2108.01374*, 2021.

[17] L. N. Ferreira, L. Mou, J. Whitehead, and L. H. Lelis, "Controlling perceived emotion in symbolic music generation with monte carlo tree search," in *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, vol. 18, no. 1, 2022, pp. 163–170.

[18] C. Bao and Q. Sun, "Generating music with emotions," *IEEE Transactions on Multimedia*, 2022.

[19] H. H. Mao, T. Shin, and G. Cottrell, "Deepj: Style-specific music generation," in *2018 IEEE 12th International Conference on Semantic Computing (ICSC)*. IEEE, 2018, pp. 377–382.

[20] W. Wang, X. Li, C. Jin, D. Lu, Q. Zhou, and Y. Tie, "Cps: Full-song and style-conditioned music generation with linear transformer," in *2022 IEEE International Conference on Multimedia and Expo Workshops (ICMEW)*. IEEE, 2022, pp. 1–6.

[21] K. Choi, C. Hawthorne, I. Simon, M. Dinculescu, and J. Engel, "Encoding musical style with transformer autoencoders," in *International Conference on Machine Learning*. PMLR, 2020, pp. 1899–1908.

[22] B. Yu, P. Lu, R. Wang, W. Hu, X. Tan, W. Ye, S. Zhang, T. Qin, and T.-Y. Liu, "Museformer: Transformer with fine- and coarse-grained attention for music generation," in *Proceedings of Advances in Neural Information Processing Systems (NeurIPS)*, 2022, pp. 1376–1388.

[23] X. Zhang, J. Zhang, Y. Qiu, L. Wang, and J. Zhou, "Structure-enhanced pop music generation via harmony-aware learning," in *Proceedings of the 30th ACM International Conference on Multimedia*, 2022, pp. 1204–1213.

[24] J. Ens and P. Pasquier, "Mmm: Exploring conditional multi-track music generation with the transformer," *arXiv preprint arXiv:2008.06048*, 2020.

[25] S. Di, Z. Jiang, S. Liu, Z. Wang, L. Zhu, Z. He, H. Liu, and S. Yan, "Video background music generation with controllable music transformer," in *Proceedings of the 29th ACM International Conference on Multimedia*, ser. MM '21. New York, NY, USA: Association for Computing Machinery, 2021, pp. 2037–2045.

[26] S. Wu, X. Li, and M. Sun, "Chord-conditioned melody choralization with controllable harmonicity and polyphonicity," *arXiv preprint arXiv:2202.08423*, 2022.

[27] A. Ramesh, P. Dhariwal, A. Nichol, C. Chu, and M. Chen, "Hierarchical text-conditional image generation with clip latents," *arXiv preprint arXiv:2204.06125*, 2022.

[28] C. Saharia, W. Chan, S. Saxena, L. Li, J. Whang, E. L. Denton, K. Ghasemipour, R. Gontijo Lopes, B. Karagol Ayan, T. Salimans *et al.*, "Photorealistic text-to-image diffusion models with deep language understanding," *Advances in Neural Information Processing Systems*, vol. 35, pp. 36 479–36 494, 2022.

[29] A. Ramesh, M. Pavlov, G. Goh, S. Gray, C. Voss, A. Radford, M. Chen, and I. Sutskever, "Zero-shot text-to-image generation," in *International Conference on Machine Learning*. PMLR, 2021, pp. 8821–8831.

[30] P. R. Cook, *Music, Cognition, and Computerized Sound: An Introduction to Psychoacoustics*. MIT press, 2001.

[31] N. S. Keskar, B. McCann, L. R. Varshney, C. Xiong, and R. Socher, "Ctrl: A conditional transformer language model for controllable generation," *arXiv preprint arXiv:1909.05858*, 2019.

[32] X. L. Li and P. Liang, "Prefix-tuning: Optimizing continuous prompts for generation," *arXiv preprint arXiv:2101.00190*, 2021.

[33] P. Liu, W. Yuan, J. Fu, Z. Jiang, H. Hayashi, and G. Neubig, "Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing," *ACM Computing Surveys*, vol. 55, no. 9, pp. 1–35, 2023.

[34] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, "Language models are few-shot learners," *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020.

[35] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: pre-training of deep bidirectional transformers for language understanding," in *NAACL-HLT (1)*. Association for Computational Linguistics, 2019, pp. 4171–4186.

[36] M. Zeng, X. Tan, R. Wang, Z. Ju, T. Qin, and T.-Y. Liu, "MusicBERT: Symbolic music understanding with large-scale pre-training," in *Findings of the Association for Computational Linguistics (ACL Findings)*, 2021, pp. 791–800.

[37] J. Ens and P. Pasquier, "Building the metamidi dataset: Linking symbolic and audio musical data." in *ISMIR*, 2021, pp. 182–188.

[38] Z. Wang, K. Chen, J. Jiang, Y. Zhang, M. Xu, S. Dai, and G. Xia, "POP909: A pop-song dataset for music arrangement generation," in *Proceedings of International Society for Music Information Retrieval Conference (ISMIR)*, 2020, pp. 38–45.

[39] J. Liu, Y. Dong, Z. Cheng, X. Zhang, X. Li, F. Yu, and M. Sun, "Symphony generation with permutation invariant language model," *arXiv preprint arXiv:2205.05448*, 2022.

[40] Y.-S. Huang and Y.-H. Yang, "Pop music transformer: Beat-based modeling and generation of expressive pop piano compositions," in *Proceedings of ACM International Conference on Multimedia (MM)*, 2020, pp. 1180–1188.

[41] A. Katharopoulos, A. Vyas, N. Pappas, and F. Fleuret, "Transformers are rnns: Fast autoregressive transformers with linear attention," in *International Conference on Machine Learning*. PMLR, 2020, pp. 5156–5165.

[42] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2014.

[43] E. Perez, F. Strub, H. De Vries, V. Dumoulin, and A. Courville, "Film: Visual reasoning with a general conditioning layer," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, 2018.

[44] M. Chen, X. Tan, B. Li, Y. Liu, T. Qin, S. Zhao, and T.-Y. Liu, "Adaspeech: Adaptive text to speech for custom voice," *arXiv preprint arXiv:2103.00993*, 2021.

A Attribute Information

Table 9 shows the detailed pre-defined musical attribute values. The value *NA* of each attribute refers to that this attribute is not mentioned in the text. Objective attributes can be extracted from MIDI files with heuristic algorithms and subjective attributes are collected from existing datasets, as shown in Table 10.

Table 9: Detailed attribute values.

Attributes	Values
Instrument	28 instruments: piano, keyboard, percussion, organ, guitar, bass, violin, viola, cello, harp, strings, voice, trumpet, trombone, tuba, horn, brass, sax, oboe, bassoon, clarinet, piccolo, flute, pipe, synthesizer, ethnic instrument, sound effect, drum. Each instrument: 0: played, 1: not played, 2: NA
Pitch Range	0-11: octaves,12: NA
Rhythm Danceability	0: danceable, 1: not danceable, 2: NA
Rhythm Intensity	0: serene, 1: moderate, 2: intense, 3: NA
Bar	0: 1-4 bars, 1: 5-8 bars, 2: 9-12 bars, 3: 13-16 bars, 4: NA
Time Signature	0: 4/4, 1: 2/4, 2: 3/4, 3: 1/4, 4: 6/8, 5: 3/8, 6: other tempos, 7: NA
Key	0: major, 1: minor, 2: NA
Tempo	0: slow (<=76 BPM), 1: moderato (76-120 BPM), 2: fast (>=120 BPM), 3: NA
Time	0: 0-15s, 1: 15-30s, 2: 30-45s, 3: 45-60s, 4: >60s, 5: NA
Artist	0-16 artists: Beethoven, Mozart, Chopin, Schubert, Schumann,J.S.Bach, Haydn, Brahms, Handel, Tchaikovsky, Mendelssohn,Dvorak, Liszt, Stravinsky, Mahler,Prokofiev, Shostakovich, 17: NA
Genre	21 genres: new age, electronic, rap, religious,international,easy listening, avant garde, RNB, latin, children, jazz, classical, comedy, pop, reggae, stage, folk, blues, vocal, holiday, country, symphony Each genre: 0: with, 1: without, 2: NA
Emotion	0-3: the 1-4 quadrant in Russell's valence-arousal emotion space 4: NA

Table 10: Extraction methods and sources of each attributes.

Type	Attribute	Extraction Method
Objective	Instrument	directly extracted from MIDI
	Pitch range	calculated based on the pitch range
	Rhythm danceability	judged with the ratio of downbeat
	Rhythm intensity	judged with the average note density
	Bar	directly extracted from MIDI
	Time signature	directly extracted from MIDI
	Key	judged with the note pitches based on musical rules
	Tempo	directly extracted from MIDI
	Time	derived from the time signature and the number of bars
Subjective	Artist	provided by a classical music dataset in MMD [36]
	Genre	provided by MAGD[14], a classical music dataset in MMD [36] and Symphony [39]
	Emotion	provided by EMOPIA [16] and the emotion-gen dataset

B Experiments

B.1 User study with baselines

In the user study, participants were provided with generated music samples along with their corresponding textual prompts. For each text description, each model (i.e., BART-base, GPT-4, MuseCoco) generated three different music clips. In each questionnaire, three samples generated with the same

text conditions were randomly picked from samples generated by BART-base, GPT-4, and MuseCoco respectively as a group. Each participant was asked to evaluate 7 groups for comparison. Three subjective metrics, musicality, controllability, and an overall score, are rated on a scale of 1 (lowest) to 5 (highest). The participants were first requested to evaluate their music profession level, as depicted in Table 11. To ensure the reliability of the assessment, only individuals with at least music profession level 3 were selected, resulting in a total of 19 participants. Secondly, they were instructed to independently evaluate two separate metrics: musicality and controllability, ensuring that scoring for one metric did not influence the other. They are also asked to give an overall score to evaluate the generated music comprehensively. For the collected results, we computed the mean and variance for each metric. The results can be found in Table 4.

Table 11: Music Profession Level

Level	Description
1	I rarely listen to music.
2	I haven't received formal training in playing or music theory, but I often listen to music and have my preferred styles, musicians, and genres.
3	I have some basic knowledge of playing an instrument or music theory, but I haven't received formal training.
4	I haven't received formal training, but I have self-taught myself some aspects such as music theory or playing an instrument. I am at an amateur level (e.g., CCOM piano level 6 or above).
5	I have received professional training in a systematic manner.

B.2 Objective Comparison with baselines

In this section, we introduce how to calculate the objective metric, the average sample-wise accuracy (ASA), in Table 4. As for MuseCoco, ten music clips are generated per prompt and we report ASA of them among the overall standard test set. Since it is labor-intensive to leverage GPT-4 with the official web page, we only guide GPT-4 to produce five music clips per prompt and calculate the ASA of 21 prompts randomly sampled from the standard test set. Besides, we utilize the released text-tune BART-base checkpoint[15] to generate five music clips per prompt and report the ASA of 44 prompts randomly chosen from the standard test set.

B.3 Text-to-attribute understanding

As shown in Table 12, all attribute control accuracy is close or equal to 100%, which indicates our model with multiple classification heads in the text-to-attribute understanding stage performs quite well.

B.4 Details of Analysis on Attribute-to-music Generation

Attribute Control Accuracy We report the control accuracy for each attribute on the test dataset, as shown in Table 13. The average attribute control accuracy of 80.42%, which provides substantial evidence for the model's proficiency in effectively controlling music generation using music attributes.

Study on Control Methods To verify the effectiveness of the control method in the attribute-to-music generations stage, we compare *Prefix Control* with two methods: *Embedding* and *Conditional LayerNorm*. For efficiency, we conducted this study on reduced-size models as follows: The backbone model of this experiment is a 6-layer Linear Transformer with causal attention. The hidden size is 512 and the FFN hidden size is 2048. The other experiment configuration is the same as Section 4.1. Since the control accuracy of objective attributes can be easily calculated, we only need to measure the controllability of each subjective attribute in listening tests. The control accuracy of each attribute is shown in Table 13. Finally, the average attribute control accuracy can be calculated based on the accuracy results from both types of attributes. To measure the controllability of subjective attributes

[15] `https://huggingface.co/sander-wood/text-to-music`

Table 12: Attribute control accuracy (%) of the text-to-attribute understanding model. I: Instrument.

Attribute	Accuracy(%)	Attribute	Accuracy(%)	Attribute	Accuracy(%)
I_piano	100.00	I_clarinet	99.92	Genre_comedy_spoken	100.00
I_keyboard	99.92	I_piccolo	99.94	Genre_pop_rock	100.00
I_percussion	100.00	I_flute	99.62	Genre_reggae	100.00
I_organ	100.00	I_pipe	100.00	Genre_stage	100.00
I_guitar	99.92	I_synthesizer	100.00	Genre_folk	100.00
I_bass	99.84	I_ethnic_instruments	99.98	Genre_blues	100.00
I_violin	99.92	I_sound_effects	99.98	Genre_vocal	100.00
I_viola	99.96	I_drum	100.00	Genre_holiday	100.00
I_cello	99.92	Genre_new_age	99.98	Genre_country	100.00
I_harp	100.00	Genre_electronic	100.00	Genre_symphony	100.00
I_strings	99.96	Genre_rap	100.00	Bar	100.00
I_voice	99.70	Genre_religious	100.00	Time Signature	100.00
I_trumpet	99.96	Genre_international	100.00	Key	100.00
I_trombone	99.94	Genre_easy_listening	100.00	Tempo	99.84
I_tuba	100.00	Genre_avant_garde	100.00	Octave	100.00
I_horn	99.94	Genre_rnb	100.00	Emotion	99.80
I_brass	100.00	Genre_latin	100.00	Time	100.00
I_sax	99.84	Genre_children	100.00	Rhythm Danceability	100.00
I_oboe	99.94	Genre_jazz	100.00	Rhythm Intensity	99.88
I_bassoon	99.96	Genre_classical	100.00	Artist	100.00

Table 13: Accuracy (%) of each attribute for attribute-to-music generation. I: Instrument.

Attribute	Accuracy(%)	Attribute	Accuracy(%)
I_piano	96.20	I_clarinet	90.63
I_keyboard	79.55	I_piccolo	86.67
I_percussion	65.19	I_flute	86.73
I_organ	80.55	I_pipe	70.73
I_guitar	91.81	I_synthesizer	78.28
I_bass	93.11	I_ethnic_instruments	77.69
I_violin	87.88	I_sound_effects	51.74
I_viola	92.03	I_drum	95.96
I_cello	86.50	Bar	71.80
I_harp	74.87	Time Signature	99.14
I_strings	86.08	Key	57.42
I_voice	75.82	Tempo	92.71
I_trumpet	84.86	Octave	61.56
I_trombone	84.64	Time	65.82
I_tuba	93.08	Rhythm Danceability	88.04
I_horn	80.13	Rhythm Intensity	80.47
I_brass	77.27	Genre	73.08
I_sax	81.74	Emotion	69.45
I_oboe	85.23	Artist	50.03
I_bassoon	90.72		

Prompt

Please combine the following sentences to one paragraph:
The music is imbued with [EMOTION]. The [KEY] key in this music provides a powerful and memorable sound. The song progresses through [NUM_BARS] bars. [TIME_SIGNATURE] is the meter of the music.

ChatGPT Response

The music is imbued with [EMOTION], creating a captivating and emotionally rich experience. It is composed in the [KEY] key, which adds a powerful and memorable sound to the composition. The song unfolds through [NUM_BARS] bars, with [TIME_SIGNATURE] serving as the meter, further enhancing its rhythmic and melodic structure.

Figure 3: ChatGPT refines concatenated templates in the prompt.

Prompt

I want a music in the ABC notation format and assign proper MIDI instrument IDs as the following:

```
X:1
T:Forest Stroll
M:3/4
L:1/4
Q:1/4=80
K:C
V:1
%%MIDI program 90 % Synthesizer
E/2F/2|G2A|G2F|E2D|C2E|F2G|A3-|A2E/2F/2|
G2A|G2F|E2D|C2E|F2G|A2F|E2D|C3||
V:2
%%MIDI program 68 % Oboe
G/2A/2|B2C|B2A|G2F|E2G|A2B|c3-|c2G/2A/2|
B2C|B2A|G2F|E2G|A2B|c2A|G2F|E3||
V:3
%%MIDI program 73 % Flute
z/2A/2|B2C|B2A|G2F|E2G|A2B|c3-|c2z/2A/2|
B2C|B2A|G2F|E2G|A2B|c2A|G2F|E3||
V:4
%%MIDI program 42 % Cello
C/2D/2|E2F|E2D|C2B,|A,2C|D2E|F3-|F2C/2D/2|
E2F|E2D|C2B,|A,2C|D2E|F2D|C2B,|A,3||
```

Compose ABC notation music as the format above according to the music description as follow: The music should feature the cello and flute. The music is in 4/4 meter. The song is 31 ~ 45 seconds in length.

GPT-4 Response

```
X:1
T:Whispering Winds
M:4/4
L:1/4
Q:1/4=80
K:C
V:1
%%MIDI program 73 % Flute
A/2B/2|c2d|e2c|A2G|F2A|B2c|d3-|d2A/2B/2|
c2d|e2c|A2G|F2A|B2c|d2A|G2F|A3||
V:2
%%MIDI program 42 % Cello
C/2E/2|F2G|A2F|C2B,|A,2C|E2F|G3-|G2C/2E/2|
F2G|A2F|C2B,|A,2C|E2F|G2C|B,2A,|C3||
```

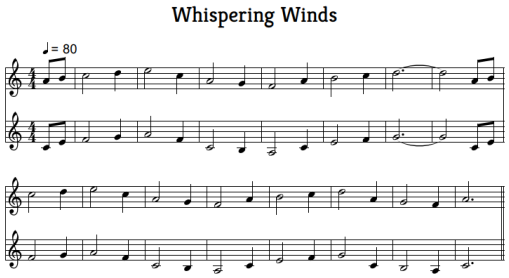


Figure 4: GPT-4 generates ABC notation tunes based on the prompt.

(such as emotion and genre), we invite 12 participants to conduct a listening test. Each participant was provided with 18 music pieces (6 pieces per control method) with corresponding subjective attributes. We asked each participant to answer: 1) Musicality(five-point scale): How similar it sounds to the music composed by a human. 2) Controllability: Does it align with the given attributes. Then we report the musicality and average attribute accuracy score in Table Table 7. The experimental results clearly demonstrate that *Prefix Control* outperforms the other two methods in terms of musicality and controllability.

B.5 Usage of GPT models

Refine texts with ChatGPT As shown in Figure 3, in order to make text descriptions more coherent and fluent, we feed concatenated templates into ChatGPT with a prompt *Please combine the following sentences to one paragraph* and then ChatGPT will give a response containing all templates within a compact paragraph.

Generate ABC notation music with GPT-4 To use GPT-4 as the baseline method for comparison, we design the instruction to guide GPT-4 as shown in Figure 4. GPT-4 can only generate symbolic music in ABC notation, so we need to explicitly point out the format. Besides, since GPT-4 can generate various ABC notation formats, some of which cannot be processed by music21, we provide an ABC notation example, teaching GPT-4 to follow its format. Meanwhile, we use the prompt, *Compose ABC notation music as the format above according to the music description as follows: [text descriptions]* to let GPT-4 generate music according to the text description. And we finally convert the ABC notations into MIDI for a fair comparison.

C Limitation

This work is mainly about generating symbolic music from text descriptions, which does not consider long sequence modeling especially. To address this, we can employ Museformer [22] as the backbone model, which proposes fine- and coarse-grained attention for handling long sequences.

The attribute set provided in this work represents only a subset of all music attributes. We aim to further explore additional attributes to offer a broader range of control options for music generation, ensuring greater diversity in the creative process.

The possibility of regenerating music based on additional text descriptions to assist users in refining their compositions is an aspect that is worth exploring.